

기계공학 전공 / 로봇 분야 신입 지원

정승호

SEUNGHO JUNG

움직이는 원리를 배우고,  
직접 만들어 보고 있습니다.

기계공학을 전공하고 로봇 개발 교육을 받고 있습니다. 로봇팔을 제작하고 센서 데이터를 다루면서, 배운 내용을 실제 동작으로 확인해 왔습니다.

3095069@naver.com

GitHub ↗



기계공학 · 로봇 제작 · 제어

## 프로젝트와 실습에서 사용한 기술

학력·교육 보기 ↗

## 01 제어·임베디드

C · STM32 · FreeRTOS  
PLC · PWM · I<sup>2</sup>C · UART  
로봇팔 제어와 펌웨어 통합

## 02 인식·데이터

Python · MATLAB  
YOLO · OpenCV · TFLite  
진동 신호 분석과 영상 인식

## 03 설계·개발 도구

SolidWorks · Ansys  
Linux · Git · Codex  
전공 실습 · AI 지원 앱 구현

# 학력과 교육, 자격 사항

기계공학에서 배운 내용을 로봇 제작과 제어 실습으로 이어가고 있습니다.

## 학력·교육

- 2026.02 졸업  
**순천향대학교 기계공학과**  
학점 3.65 / 4.5
- 2026.04.14 - 2026.11.09 교육 중  
**대한상공회의소 서울기술교육센터**  
AI융합 로봇 SW개발자 3기  
  
Python·Linux 기반 AI 프로그래밍, ROS/ROS2 통신·제어, Cortex-M·PLC와 산업용 네트워크를 학습하고 있습니다.
- 2019.02 졸업  
**대성고등학교**

2026년 9월 기준 · 로봇 개발 교육 과정은 수료 전입니다.

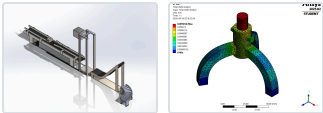
## 자격·어학

**초경량비행장치조종자**  
2019.09 · 한국교통안전공단

**운전면허 1종 보통**  
2019.08 · 경찰청

**TOEIC 705**  
2025.12

설계·해석은 전공 실습에서, 앱 구현은 AI 도구와 함께 익혔습니다.



**설계·해석 실습**  
이미지 4개 보기 ↗

# 일주일 만에 완성한 이동형 로봇

4륜 이동체에 로봇팔을 결합하고, 스마트폰에서 함께 조작하도록 만들었습니다.



실제 제작물 · 회전 관절 5개와 그리퍼

고정된 로봇팔로 닿기 어려운 곳까지 이동해 물체를 집고 옮기는 시제품입니다. 앱에서 주행과 관절을 조작하고, 저장한 동작을 다시 재생할 수 있습니다.

## 내가 맡은 일

5인 팀의 조장을 맡아 로봇팔을 제작·제어하고 주행·로봇팔 펌웨어를 통합했습니다. 통신 규격과 주요 설정을 팀원들과 먼저 맞춘 뒤 앱 조정에 참여했습니다.

[STM32F411](#)[C · FreeRTOS](#)[Flutter](#)[Bluetooth](#)[▶ 시연 영상 보기](#)[소스 코드 ↗](#)

제작 시기

2026.07

제작 기간 · 팀 구성

1주 · 5인 팀

구동 구성

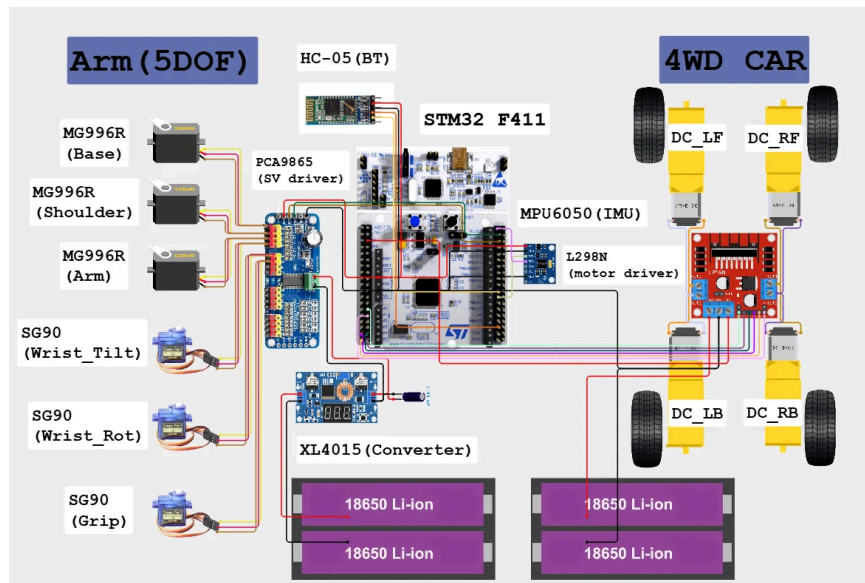
4WD · 서보 6개

구현 범위

수동 조작 · 티칭

# 앱의 명령이 동작으로 이어지도록

통신 규격을 맞추고, 주행·로봇팔·저장 기능의 실행 조건을 나누었습니다.



발표자료의 전체 배선도

**공동 구현 사양** 앱 정지는 논리적 출력 차단입니다. 공동 전원을 끊는 물리 비상정지 장치는 적용하지 않았습니다.

## 01 16바이트 명령·응답

주행, 로봇팔, 티칭, 설정을 모드로 구분합니다. 앱의 요청과 펌웨어의 응답을 연결해 처리 상태를 확인합니다.

## 02 PCA9685와 동작 저장

서보 6개를 제어합니다. 티칭·설정은 Flash에 저장하며 최대 12개 시퀀스 × 30개 웨이포인트를 지원합니다.

## 03 동작이 겹치지 않도록

로봇팔 이동과 Flash 작업 중에는 주행을 잠급니다. 유효한 주행 명령이 500ms 동안 없으면 차량을 정지시킵니다.



# 조작 화면과 로봇의 움직임을 함께 다듬었습니다

주행·관절 제어·원점 보정·티칭을 한 앱에 담고, 구동 중 나타난 문제를 나눠 개선했습니다.

## 01 주행



앱 TEST 모드 화면

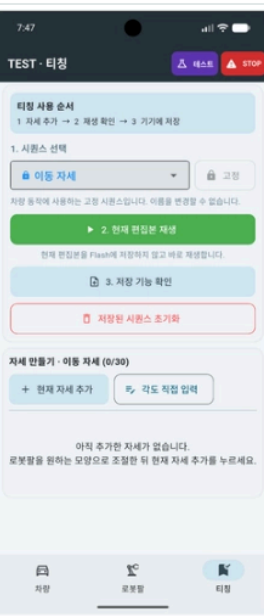
## 02 로봇팔



## 03 원점 보정



## 04 티칭



주행·관절·보정·티칭

**남은 과제** 엔코더 피드백, 역기구학 XYZ 제어, 물리 비상정지와 전원 보호는 추가 구현이 필요합니다.

## 직접 해결·전원과 모터

## 부하가 걸릴 때 생긴 떨림

전원부에 커패시터를 추가하고 토크가 높은 모터로 교체해 해결했습니다.

## 공동 구현·관절 제어

## 갑작스러운 관절 움직임

20ms 주기의 S-curve로 이동하도록 구성했습니다. 서보의 실제 위치 피드백은 없습니다.

## 팀원 담당·통합 참여

## 직진 중 틀어지는 방향

IMU 상대 Yaw로 좌우 출력을 보정하고, 최대 220 PWM으로 제한해 보정 여유를 남겼습니다.

# 보행 영상에서 장애물과 지면 찾기

라즈베리파이5에 객체 탐지와 지면 분할 모델을 적용한 팀 프로젝트입니다.



녹화 영상을 입력으로 사용한 추론 시연

영상 속 사람·차량·장애물을 찾고, 인도와 차도 같은 지면을 구분했습니다. 보행 경로에 가까운 대상을 판단해 음성 알림으로 연결했습니다.

## 내가 맡은 일

학습 데이터의 라벨을 통합하고 모델 학습·변환과 실행 구성을 담당했습니다. 보행·차량 ROI와 알림 판단 로직을 구성해 녹화 영상으로 동작을 확인했습니다.

Python · OpenCV

YOLO26n

TFLite INT8

▶ 시연 영상 보기

소스 코드 ↗

수행 기간

2026.05.11-05.21

팀 구성

4인

객체 라벨

29개 → 10개

지면 라벨

23개 → 7개

## 제한된 장치에서 실행하기 위한 조정

해상도와 모델 형식을 바꾸고, 알림에 필요한 영역을 따로 정했습니다.

### 01 연산 부담을 줄였습니다

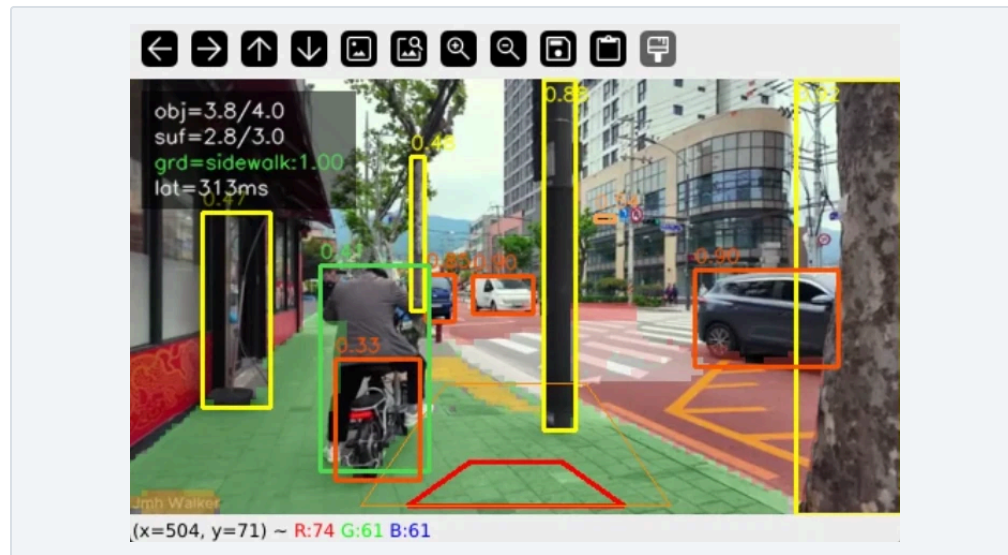
입력 영상은  $1920 \times 1080$ 에서  $512 \times 288$ 로 줄이고, 모델을 INT8 TFLite로 변환했습니다.

### 02 보행·차량 영역을 나눴습니다

보행 ROI는 발 앞쪽에 해당하는 하단 중앙에 뒀습니다. 옆에서 접근하는 차량은 더 넓은 ROI로 판단했습니다.

### 03 녹화 영상으로 시험했습니다

영상을 재생하는 동안 모델이 추론하도록 구성했습니다. 객체는 4fps, 지면은 3fps로 추론 주기를 제한했습니다.



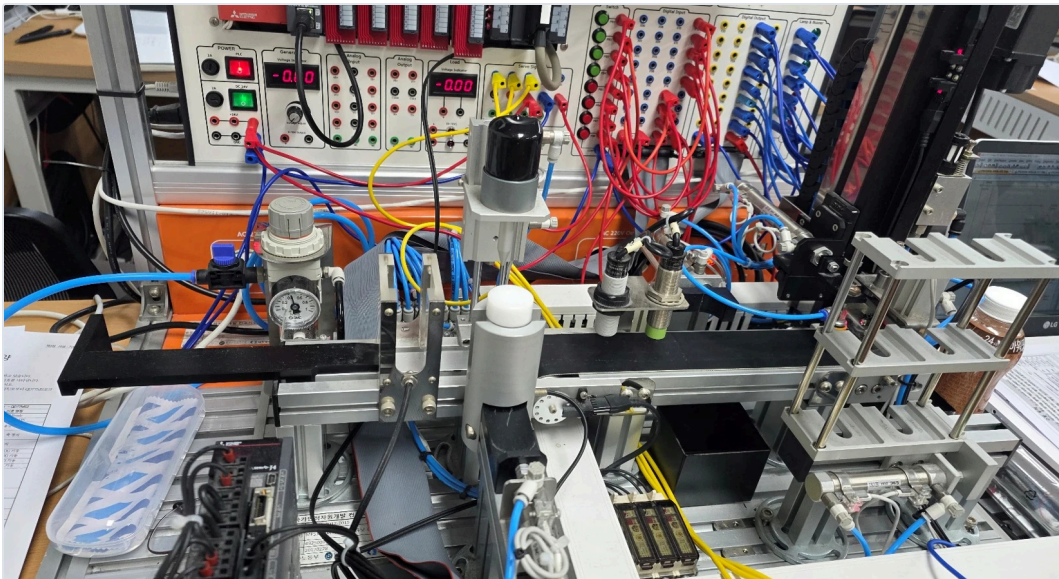
빨간색: 보행 경로 · 주황색: 차량 접근 영역

4fps·3fps는 설정한 제한값입니다. 실측 최대 처리 속도를 뜻하지 않습니다.

**확인한 한계** 보행자 신호등 인식이 부족했고, 역광과 작은 객체에서 판단이 어려웠습니다. 실제 보행 안전성을 검증한 제품은 아닙니다.

# 공급부터 적재까지 공정을 연결했습니다

MPS 트레이너에서 소재 분류와 1-4층 창고 적재·비움 동작을 구현했습니다.



프로젝트에 사용한 PLC·모션·컨베이어 장비

컨베이어와 실린더, 서보모터를 이용해 공급·가공·분류·저장을 연결했습니다. 여러 장치가 병렬로 움직이면서 수동으로도 개별 제어할 수 있게 구성했습니다.

## 내가 맡은 일

병렬 처리 로직과 HMI 화면 작성을 맡았습니다. 팀원의 수동 제어 코드와 통합하는 과정을 돕고, 메모리 상태 때문에 생기는 오동작을 디버깅했습니다.

Mitsubishi Q PLC

GX Works2

GT Designer3

▶ 시연 영상 보기

수행 시기

2026.06 · 3일

팀 구성

2인

제어기

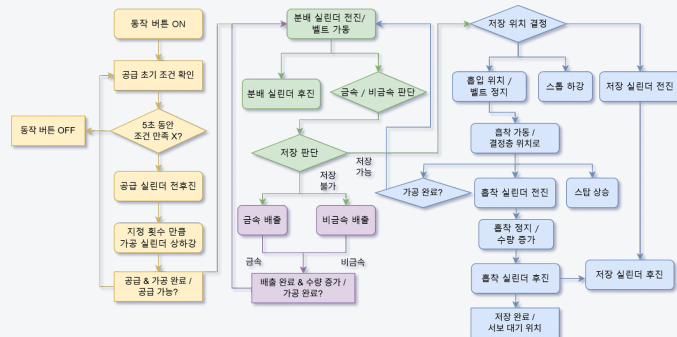
Q03UDVCPU

모션 모듈

QD77MS2

**동시에 움직여도 상태가 섞이지 않도록**

저장 가능 여부를 판단하는 메모리와 실제 적재 정보를 분리했습니다.



발표자료의 병렬 시퀀스 구성도



자동 운전·적재 상태



## 위치·속도 티칭

01 상태가 다음 동작에 남았습니다

메모리 저장·초기화 시점이 맞지 않으면 이전 동작이 다음 시퀀스의 판단에 영향을 줍니다.

02 조건과 완료 시점을 맞췄습니다

판단용·적재용 메모리를 나누고 동작 조건과 완료 시점에 맞춰 저장·초기화하도록 수정했습니다.

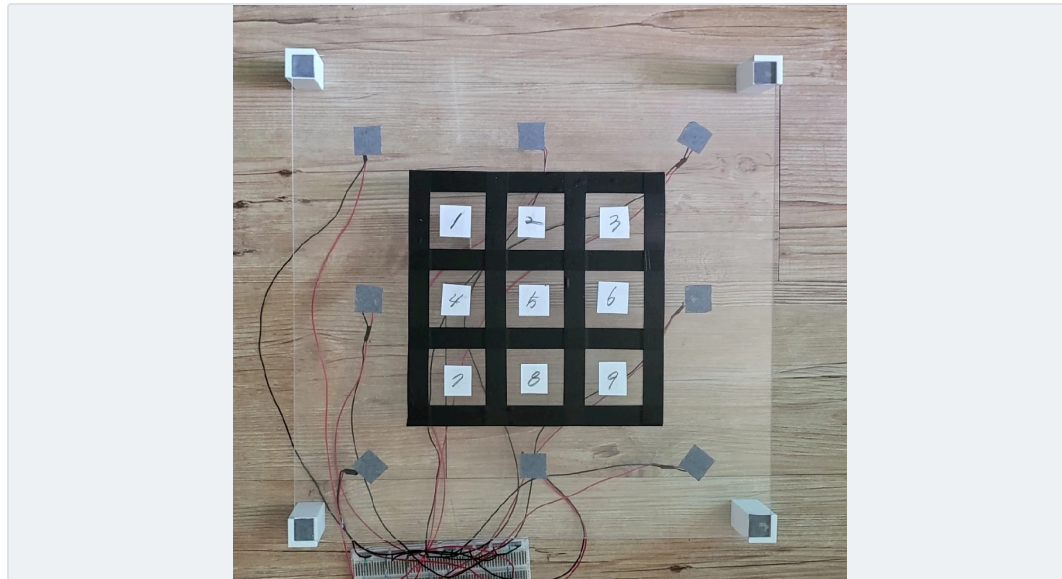
### 03 통합할 때도 주소를 맞춰줍니다

조건별 동작을 확인하고, 팀원 코드와 겹치는 메모리 주소를 정리했습니다. 이후 작업에서는 사용 번지를 먼저 정해야 한다는 점을 배웠습니다.



# 노크한 위치를 진동 신호로 구분하기

센서 간 시간 차와 시간·주파수 특징을 입력으로 사용해 기계학습 모델을 비교했습니다.



400×400mm 아크릴판 · 직접 구성한 실험 장치

아크릴판을 두드릴 때 생기는 진동을 8개의 피에조 센서로 수집했습니다.  
그 신호로 노크가 발생한 9개 구역을 구분하는 연구입니다.

## 내가 맡은 일

연구 방향과 데이터 처리·모델 비교를 주도했습니다. MATLAB으로  
전처리·특징 추출·실시간 탐지 프로그램을 구현했습니다.

MATLAB

FFT · 신호 처리

K-NN · SVM · ANN

▶ 시연 영상 보기

논문 표기 시점

2023.05

연구 구성

6인 공동 연구

수집 데이터

9구역 × 140회

기록 조건

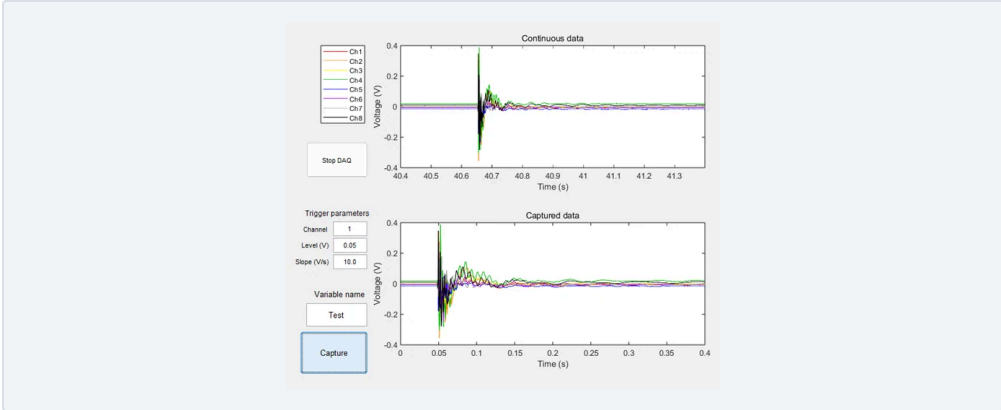
2,500Hz · 0.4초

# 입력 특징에 따라 결과가 달랐습니다

같은 아크릴판에서 얻은 데이터를 학습 70%, 검증 30%로 무작위 분할해 비교했습니다.

## 신호 수집부터 실시간 탐지까지

61Hz 하이패스 필터와 데이터 평탄화를 적용한 뒤, 특징을 추출·정규화해 모델을 학습했습니다.



연속 신호와 트리거 기록 화면

학습한 K-NN을 위치 탐지 프로그램에 적용해 노크 위치를 표시하고 데이터를 기록했습니다.

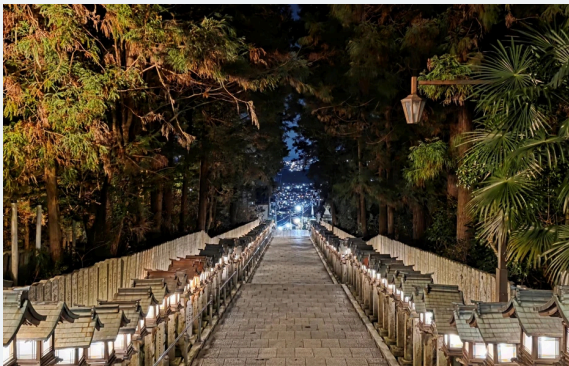
## K-NN 검증 정확도

| 입력 특징        | 정확도     |
|--------------|---------|
| 센서 간 피크 시간 차 | 90.741% |
| 시간 영역 특징     | 96.296% |
| 주파수 영역 특징    | 97.884% |

동일 아크릴판에서 K-NN의 입력 특징을 비교한 값입니다. 전체 모델의 최고값은 아니며, 다른 재질·센서 배치에서는 별도 검증이 필요합니다.

# 여행 사진과 이동 경로를 함께 보기

사진을 찍었던 순간을 이동 기록과 함께 돌아보고 싶어 웹앱을 만들었습니다.



직접 촬영한 기획 배경 사진입니다. 앱 실행 화면은 아닙니다.

## 만든 기능

Google Timeline과 사진·영상을 연결했습니다. 이동 경로 재생과 사진 여정 모드를 나누고, PC와 모바일 화면에 맞춰 배치를 바꾸도록 구성했습니다.

[React · TypeScript](#)[MapLibre](#)[Node.js](#)

## 직접 결정한 부분

화면과 기능을 구상하고 Codex로 구현했습니다. 사용 중 발견한 오류와 불편을 정리해 수정할 동작과 원하는 결과를 구체적으로 요청했습니다.

[소스 코드 ↗](#) [기능·실행 안내 ↗](#)



# 기본 기능 다음에는 사용성을 다듬었습니다

기본 구현보다 세부 기능과 사용성을 다듬는 데 더 많은 시간이 들었습니다.

01

## 이동 기록·사진

Timeline JSON과  
사진·영상 정보 입력

02

## 경로·장면 구성

촬영 시간·위치 연결  
여행 선택과 경로 계획

03

## 경로·사진 재생

모드별 재생 위치 유지  
화면 크기에 맞춘 배치

저장소 문서를 바탕으로 정리한 기능 흐름입니다.

## 세부 동작 수정

이를 보완하기 위해 앱에서 찾은 오류와 원하는 동작을 정리해 AI에 수정을 요청했습니다.

## 기록이 불확실할 때

이동 기록이 끊기거나 GPS와 위치가 크게 다르면 이동수단·상세 장소를 단정하지 않도록 처리했습니다.

## 개인 데이터 처리

사진과 Timeline 원본을 외부에 올리지 않는 로컬 우선 방식입니다. 지도 타일은 외부에서 받을 수 있습니다.

**검증 기록** 저장소에 타입 검사·단위 테스트·Chromium 검증 기록이 있습니다. 원본 미디어 전체와 실기기 검증은 별도입니다.